	Result	Time	Cycles	GPU	SM Frequency	Process	Attributes
Current	7534 - swiglu_kernel (3, 1, 1)x(256, 1, 1)	2.30 usecond	3,651	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.58 cycle/nsecond	[11435] run	

GPU Speed Of Light Throughput

GPU Throughput Rooflines

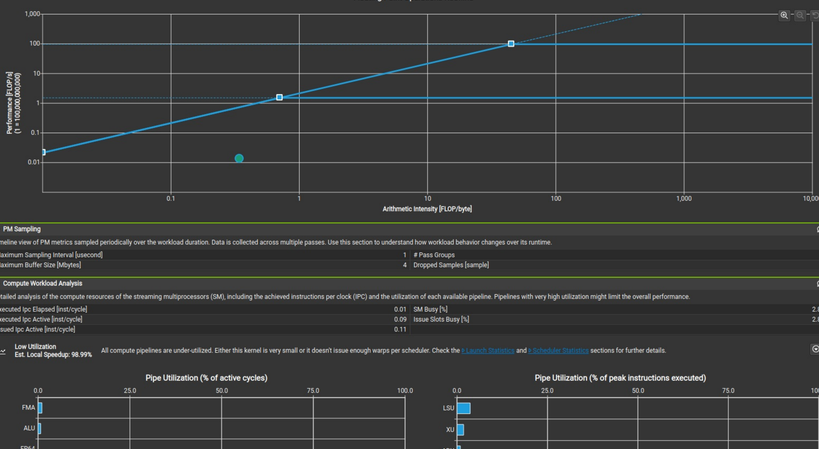
High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	0.22	Duration [usecond]	2.30
Memory Throughput [%]	1.81	Elapsed Cycles [cycle]	3,651
L1/TEX Cache Throughput [%]	4.06	SM Active Cycles [cycle]	221.92
L2 Cache Throughput [%]	0.67	SM Frequency [cycle/nsecond]	1.58
DRAM Throughput [%]	1.81	DRAM Frequency [cycle/nsecond]	6.72

Small Grid This kernel grid is too small to fill the available resources on this device, resulting in only 0.0 full waves across all SMs. Look at ℹ Launch Statistics for more details.

Roofline Analysis The ratio of peak float (fp32) to double (fp64) performance on this device is 64:1. The kernel achieved close to 0% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the 🌐 Kernel Profiling Guide for more details on roofline analysis.

Floating Point Operations Roofline



PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [usecond]	1	# Pass Groups	2
Maximum Buffer Size [Mbytes]	4	Dropped Samples [sample]	0

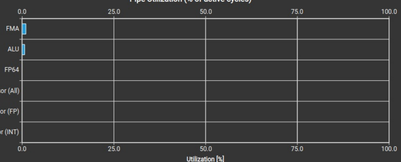
Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

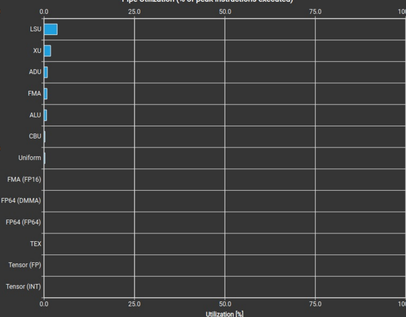
Executed Ipc Elapsed [inst/cycle]	0.01	SM Busy [%]	2.82
Executed Ipc Active [inst/cycle]	0.09	Issue Slots Busy [%]	2.82
Issued Ipc Active [inst/cycle]	0.11		

Low Utilization **Est. Local Speedup: 98.99%** All compute pipelines are under-utilized. Either this kernel is very small or it doesn't issue enough warps per scheduler. Check the ℹ Launch Statistics and ℹ Scheduler Statistics sections for further details.

Pipe Utilization (% of active cycles)



Pipe Utilization (% of peak instructions executed)



Memory Workload Analysis

Memory Tables

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/second]	3.89	Mem Busy [%]	0.67
L1/TEX Hit Rate [%]	33.33	Max Bandwidth [%]	1.81
L2 Hit Rate [%]	44.43	Mem Pipes Busy [%]	0.22
L2 Compression Success Rate [%]	0	L2 Compression Ratio	0

Shared Memory

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	0	0			
Shared Load Matrix	0	0	0	0	0
Shared Store	0	0	0	0	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	24	0.03	0
Total	0	0	24	0.03	0

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2	Returns to SM
Local Load	0	0	0	0	0	0	0	0			
Global Load	48	48			192	4	0	6,144	192	0.22	
Global Load To Shared Store (access)	0	0	48	0.05	0	0		0	0		
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0	
Surface Load	0	0	0	0	0	0	0	0	0	0	
Texture Load	0	0	0	0	0	0	0	0	0	0	
Global Store	24	24	24	0.03	96	4	100	3,072			
Local Store	0	0	0	0	0	0	0	0	96	0.11	
Surface Store	0	0	0	0	0	0	0	0	0	0	
Global Reduction	0	0	0	0	0	0	0	0	0	0	
DSMEM Reduction							-		0	0	
Surface Reduction	0	0	0	0	0	0	0	0	0	0	
Global Atomic ALU	0										
Global Atomic CAS	0	0	0	0	0	0	0	0	0	0	see
Surface Atomic ALU	0										
Surface Atomic CAS	0	0	0	0	0	0	0	0	0	0	see
Loads	48	48	48	0.05	192	4	0	6,144	192	0.22	
Stores	24	24	24	0.03	96	4	100	3,072	96	0.11	
Atomics & Reductions	0	0	0	0	0	0	0	0	0	0	

L2 Cache

	Requests	Sectors	Sectors/Req	% Peak	Hit Rate	Bytes	Throughput	Sector Misses to Device	Sector Misses to System	Sector Misses to Peer
L1/TEX Load	48	192	4	0.17	0	6,144	2,666,666,666.67	192	0	0
L1/TEX Store	24	96	4	0.17	100	3,072	1,333,333,333.33	0	0	0
L1/TEX Atomic ALU	0	0	0	0	0	0	0	0	0	0
L1/TEX Atomic CAS	0	0	0	0	0	0	0	0	0	0
L1/TEX Reduction	0	0	0	0	0	0	0	0	0	0
L1/TEX Total	72	288	4	0.25	33.33	9,216	4,000,000,000	192	0	0
ECC Total		0	-	0		0	0	0		
GPU Total	169	763	4.51	0.67	62.55	24,416	10,597,222,222.22	197	0	0

L2 Cache Eviction Policies

	First	Hit Rate	Last	Hit Rate	Normal	Hit Rate	Normal Demote	Hit Rate
L1/TEX Load	0	0	0	0	192	0	0	0
L1/TEX Store	0	0	0	0	96	100	0	0
L1/TEX Atomic	0	0	0	0	0	0	-	-
L1/TEX Total	0	0	0	0	288	33.33	0	0
GPU Total	157	100	0	0	477	58.70	0	0

Device Memory

	Sectors	% Peak	Bytes	Throughput
Load	280	1.81	8,960	3,888,888,888.89
Store	0	0	0	0
Total	280	1.81	8,960	3,888,888,888.89

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	2.31	No Eligible [%]	96.56
Eligible Warps Per Scheduler [warp]	0.04	One or More Eligible [%]	3.44
Issued Warp Per Scheduler	0.03		

Issue Slot Utilization **Est. Local Speedup: 96.56%** Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 29.1 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this kernel allocates an average of 2.31 active warps per scheduler, but only an average of 0.04 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, reduce the time the active warps are stalled by inspecting the top stall reasons on the ℹ Warp State Statistics and ℹ Source Counters sections.

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	67.12	Avg. Active Threads Per Warp	32
Warp Cycles Per Executed Instruction [cycle]	83.90	Avg. Not Predicated Off Threads Per Warp	30.40

Imc Miss Stalls **Est. Speedup: 42.61%** On average, each warp of this kernel spends 28.6 cycles being stalled waiting for an immediate constant cache (IMC) miss. A read from constant memory costs one memory read from device memory only on a cache miss; otherwise, it just costs one read from the constant cache. Immediate constants are encoded into the SASS instruction as c[bank][offset]. Accesses to different addresses by threads within a warp are serialized, thus the cost scales linearly with the number of unique addresses read by all threads within a warp. As such, the constant cache is best when threads in the same warp access only a few distinct locations. If all threads of a warp access the same location, then constant memory can be as fast as a register access. This stall type represents about 42.6% of the total average of 67.1 cycles between issuing two instructions.

Long Scoreboard Stalls **Est. Speedup: 30.47%** On average, each warp of this kernel spends 20.5 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 30.5% of the total average of 67.1 cycles between issuing two instructions.

Warp Stall Check the ℹ Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The 🌐 Kernel Profiling Guide provides more details on each stall reason.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	480	Avg. Executed Instructions Per Scheduler [inst]	5
Issued Instructions [inst]	600	Avg. Issued Instructions Per Scheduler [inst]	6.25

FP32 Non-Fused Instructions **Est. Speedup: 0.51%** This kernel executes 0 fused and 96 non-fused FP32 instructions. By converting pairs of non-fused instructions to their 🌐 fused, higher-throughput equivalent, the achieved FP32 performance could be increased by up to 50% (relative to its current performance). Check the Source page to identify where this kernel executes FP32 instructions.

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	3	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	16	Static Shared Memory Per Block [byte/block]	0
Block Size	256	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	768	Driver Shared Memory Per Block [byte/block]	1.02
Waves Per SM	0.02	Shared Memory Configuration Size [Kbyte]	16.38
Uses Green Context	0	# SMs [SM]	24

Small Grid **Est. Speedup: 87.50%** The grid for this launch is configured to execute only 3 blocks, which is less than the GPU's 24 multiprocessors. This can underutilize some multiprocessors. If you do not intend to execute this kernel concurrently with other workloads, consider reducing the block size to have at least one block per multiprocessor or increase the size of the grid to fully utilize the available hardware resources. See the 🌐 Hardware Model description for more details on launch configurations.

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	16
Theoretical Active Warps per SM [warp]	48	Block Limit Shared Mem [block]	16
Achieved Occupancy [%]	13.68	Block Limit Warps [block]	6
Achieved Active Warps Per SM [warp]	6.57	Block Limit SM [block]	24

Achieved Occupancy **Est. Speedup: 86.32%** The difference between calculated theoretical (100.0%) and measured achieved occupancy (13.7%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the 🌐 Occupancy Best Practices Guide for more details on optimizing occupancy.

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	221.92	Average L1 Active Cycles [cycle]	221.92
Average L2 Active Cycles [cycle]	525.31	Average SMSP Active Cycles [cycle]	181.83
Average DRAM Active Cycles [cycle]	280	Total SM Elapsed Cycles [cycle]	87,560
Total L1 Elapsed Cycles [cycle]	87,560	Total L2 Elapsed Cycles [cycle]	56,944
Total SMSP Elapsed Cycles [cycle]	350,240	Total DRAM Elapsed Cycles [cycle]	61,952

SMs Workload Imbalance **Est. Speedup: 5.34%** One or more SMs have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 87.71% above the average, while the minimum instance value is 100.00% below the average.

L1 Slices Workload Imbalance **Est. Speedup: 5.34%** One or more L1 Slices have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 87.71% above the average, while the minimum instance value is 100.00% below the average.

L2 Slices Workload Imbalance **Est. Speedup: 7.04%** One or more L2 Slices have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 47.68% above the average, while the minimum instance value is 92.00% below the average.

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	48	Branch Efficiency [%]	0
Branch Instructions Ratio [%]	0.10	Avg. Divergent Branches	0